I. Integration and Test Results

To ensure all engineering efforts towards both the SCADA system and the DCS system would be compatible for simultaneous testing and use the two systems were continuously tested together during development allowing refinements on both fronts to be made. The DCS system was developed in hardware and was tested using temporary Arduino C++ code that would be used to calibrate the system and inform the development of C++ code-generation. The tests run on each subsystem as well as the resulting information will be separated by subsystem.

A. Hardware Development and Testing

Before testing any control circuitry, verification of the operation of the valves and pumps was performed using direct 12 V separated by a Single Pole Single Throw enable switch to turn the pumps on and off. A 12V 2A AC Adapter Power supply was used to power the actuators and was verified to have enough current to power 3 Actuators at once. From these tests, operation of the pump's intake and release from one tank to another was verified before being connected to any control circuitry.

1) Issues with Power Distribution: In the initial design, N-channel MOSFETs, pull-down resistors, and flyback diodes to switch inductive load were used to drive relays responsible for switching 12 V loads such as a solenoid valve and water pumps. However, this first iteration exhibited inconsistent behavior during testing. The relays failed to actuate reliably due to unstable gate drive and grounding issues, and in some cases only one 12 V load could be operated at a time. This indicated limitations in both the control circuitry and overall power distribution.

2) Debugging Power Supply Issues: To address these issues, a commercially available buck converter was introduced to regulate and step down the supply voltage for logic-level components, providing a more stable voltage source. While the converter was capable of supplying up to 2 A under max load, the increased complexity of the power distribution network introduced additional risk during testing. Although relay actuation became more consistent, the system still failed to reliably switch the 12 V loads, indicating that the underlying issue was not solely related to supply regulation.

The relays were intended to electrically isolate the 5 V logic domain from the 12 V actuator domain; however, the discrete implementation did not consistently achieve this objective. These limitations ultimately led to a redesign of the actuation stage using a commercially available AEDIKO DC relay module with proper integrated driver circuitry and protection components such as optocoupler isolation that provides driving ability and stable performance.

This modification simplified the circuit design while providing quality electrical isolation between the low-voltage logic control signals (5V) and high-power components (12V). Similarly the inclusion of internal protection elements such as flyback diodes and optocouplers, reduced voltage transients and improved switching stability while the onboard LEDs showcased the on-off states of the relay when providing power for the pumps. These alterations were able to enhance the SCADA system by improving robustness and reducing system complexity compared to integrating a custom relay PCB with limited knowledge of power electronics.

3) Worst Case Current Draw: In terms of powering all components in the project, a table below was created to determine the worst case for current draw of all components in the system. From this tabulation and consulting Arduino datasheet, it was determined that the total current of the system was too high for the Arduino to support, thus introducing the need for a dedicated 5V 1A AC adapter for the relays separate from the Arduino's 5V supply.

TABLE I
Current draw of system components.

Part	I (mA)
Relay 1 ON	80
Relay 2 ON	80
Relay 3 ON	80
Flow Rate Sensor 1	10
Flow Rate Sensor 2	10
Water Level Sensor	5.2
Digital Output 5	5
Digital Output 6	5
Digital Output 7	5
Potentiometer	5
Total without Relays	45.2
Total with Relays	285.2
Maximum I of Arduino	150
Maximum I of Alt. Power Supply	1000

4) Testing of Water Level Sensor: Initial designs included both a binary water level sensor on each tank to indicate when a tank was full as well as an analog sensor for one main tank. This scope was then scaled down to just focus on a high quality Analog water level sensor as this sensor would act as the main feedback of the system. The water level sensor chosen was the eTape 12" Continuous Fluid Level Sensor PN-12110215TC-X. Our methodology while using the sensor included both multimeter testing of the resistance of the sensor in response to water level change and serial printing the analog value in the Arduino IDE serial terminal. Through repeated measurements we determined that the water level resistance ranged from 400 ohms to 2.2kOhms. For our purposes we primarily used resistances ranging from 1.2kΩ to 2.2kΩ.

This is consistent with the datasheet provided by the manufacturer as the tank we were using was 8" deep therefore only using about 75% of the total length of the sensor. This limited accuracy in the change of water level data but gave us more modulability if we were to change tank size. The output voltage sent to the analog input of our Arduino was obtained through a voltage divider of a known 560 Ω resistor and the water level sensor using the following equation, followed by

a simple example when 5V is input:

$$V_{analog} = Vin \cdot \frac{R_{sense}}{R_{sense} + 560\ \Omega} \tag{1}$$

$$V_{analog} = 5\ \mathbf{V} \cdot \frac{2.2\ k\Omega}{2.2\ k\Omega + 560\ \Omega} \approx 4.07\ \text{V} \tag{2}$$

This voltage was mapped onto Arduino analog values of about 5mV per analog unit. This calculation is based on the analog to digital step size of the Arduino. From testing of the water level sensor while connected to the Arduino IDE Serial Terminal, we obtained consistent analog values of 750 to 824 which when mapped onto the 5mV per analog unit is a range of 3.66 V to 4.03V using the following equation, followed by a simple example when 5V is input:

$$V_{analog} = \text{ADC}_{val} \cdot \frac{V_{in}}{1024} \tag{3}$$

$$V_{analog} = 824 \cdot \frac{5\ \text{V}}{1024} \approx 4.02\ \text{V} \tag{4}$$

To confirm this finding, these calculations can be run in the reverse direction, returning the original input voltage, shown below. From testing both physical resistance from multimeter and serially printed Analog Values we were able to verify and calculate the minimum and maximum ranges of our system's water level.

$$V_{in} = V_{analog} \cdot \frac{560\Omega + R_{sense}}{R_{sense}} \tag{5}$$

$$V_{in} = 4.02\ \text{V} \cdot \frac{560\Omega + 2.2\ k\Omega}{2.2\ k\Omega} \approx 5.04\ \text{V} \tag{6}$$

5) Testing of Flow Rate Sensor: While not used in the final implementation of the project, a flow rate sensor was also used which sent variable length pulses to the Arduino's digital input based on the rate of water flowing through the sensor. In order to verify results of the flow rate sensor and operational code we took timed trials of the system filling a 1.5 Liter bottle and divided the volume of the bottle by the time it took to fill the bottle in minutes to get the Liters per minute flow rate of our pumps. We verified that the pumps were operating far below the advertised 4 L/min and were consistently running at 0.78 L/min which was consistent with the numbers observed on the Arduino IDE Serial Terminal.

B. Trouble Shooting Test Control Code

Before integration of the initial Arduino C++ code with our water tank system, a simple Analog Potentiometer and LED indicators were used to represent the Water Level Sensor and Digital Outputs to Relays. This allowed for prototyping the code away from the main system. Modifications were added to the code including software delay functions, and increasing the tolerance for the analog value to reach the desired setpoint. Using minimum and maximum values obtained from testing of the water level sensor, a preliminary control code was constructed from this prototype that could interface with the water tank system.

Upon using the above mentioned C++ Arduino control code with our water tank system, noticeable flickering between states would occur and would be exacerbated by the ripple of the water due to the pumps. The sensitivity of the water level sensor was discovered to be too high especially when crossing the threshold to the setpoint's tolerance. To remedy this, a moving average function was added to the code which filtered out the flickers caused by the slight ripple of the water. Additionally an incrementing counter was added to allow the pump to remain on long enough for the water level to cross into the setpoint region before the system would shut it off. This reduced response time but allowed the system to become more stable instead of oscillating between shutting the pump on and off frequently which put strain on our hardware.

During testing the pump responsible for draining one tank was discovered to create a siphon that would continuously drain water from the tank even when turned off, to remedy this a Digitally controlled valve was added in line with the pump and was added as another Digital Output for the code, logically tied to the drain pump's digital output.

C. SCADA Software Development

The SCADA software, which was primarily developed by Brandon with experimental assistance from Karina, was tested at the addition of each feature (of which there are dozens, a full list of software features can be found in the help menu when the CLI is used) by connecting an Arduino board to the Framework 16 laptop and evaluating the new feature's performance and functionality. This was developed in much the same way most code is, through trial and error, and debugging, as such there is not one significant experiment that occurred like there was for hardware testing.

When C++ code generation was implemented it was initially non-functionally buggy, the testing that resulting in that being fixed incolved git-pulling the new code onto the Raspberry Pi v4, attempting code-generation and flashing, it fails. A USB is mounted on the Raspberry Pi (through SSH), the ino file generated by the SCADA software is copied onto the drive, then the drive is unmounted. Then the code is inspected on the laptop in the Arduino IDE. When an error is resolved the code-generator is updated, the code is pushed to GitHub and the cycle repeats. This activity took around 12 hours and was done in one sitting, but the code generator works.